

Python p/ Processamento de Dados

Etapa 6

Professor(a):

E-mail: marcelo.hama@prof.infnet.edu.br



Marcelo Tomio Hama

Head de Engenharia de Software/TI

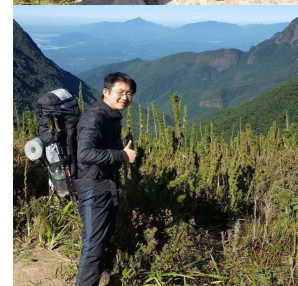
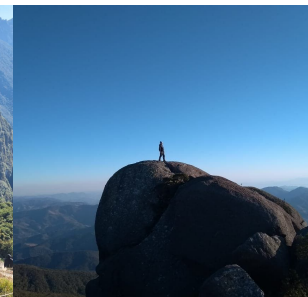
Prof. MSc Instrutor de Ensino Superior

LinkedIn: [marcelohama](#) | Facebook: [marcelo.hama](#) | Site: <http://mthama.eti.br/>

FORMAÇÃO

- BSc em Ciência da Computação - Univ. Estadual Paulista;
- MSc em Computação - Univ. Federal do Rio Grande do Sul;
- MBA Executivo - FIA Business School.

EXPERIÊNCIA PROFISSIONAL



Manipulação de arquivos e JSON

Python oferece duas abordagens complementares para trabalhar com dados persistidos.

- Arquivos em disco são manipulados via função nativa, com modos específicos para cada tipo de manipulação. O padrão recomendado é sempre usar um gerenciador de contexto para garantir o fechamento automático do arquivo mesmo em caso de erro. Para operações mais avançadas como copiar, mover e renomear, existem módulos complementares;
- Dados JSON são manipulados por outro módulo nativo, com quatro funções principais para ler/escrever em strings e arquivos. O mapeamento entre os dois mundos é direto, onde dicionários viram objetos, listas viram vetores, *None* vira *null*. Tipos não suportados nativamente, como *datetime*, exigem um serializador customizado via parâmetro default.

Na prática, as duas abordagens se combinam com frequência: abertura de arquivo e carregamento de JSON.

Manipulação de arquivos e JSON

PARTE 1

Manipulação de Arquivos em Disco

Manipulação de Arquivos em Disco

Entrada e Saída de Arquivos

O tipo mais simples de persistência é um arquivo comum, às vezes chamado de arquivo plano (flat file). Você lê de um arquivo para a memória e escreve da memória para um arquivo.

Criar ou Abrir com *open()*

Chamar a função *open* antes de realizar as seguintes operações:

- Ler um arquivo existente;
- Escrever em um novo arquivo;
- Acrescentar conteúdo a um arquivo existente;
- Sobrescrever um arquivo existente.

Sintaxe do *open*

fileobj = *open*(*filename*, *mode*)

Manipulação de Arquivos em Disco

Criar ou Abrir com *open()*

- ***fileobj*** é o objeto de arquivo retornado por ***open()***;
- ***filename*** é o nome do arquivo (tipo *string*);
- ***mode*** é uma *string* que indica o tipo do arquivo e o que fazer com ele.
 - A primeira letra do *mode* indica a operação:
 - ***r*** para ler;
 - ***w*** para escrever. Se o arquivo não existir, ele será criado. Se o arquivo já existir, ele será sobrescrito;
 - ***x*** para escrever, mas somente se o arquivo ainda não existir;
 - ***a*** para anexar (escrever após o final) se o arquivo já existir.
 - A segunda letra de *mode* indica o tipo do arquivo:
 - ***t*** (ou nada) significa texto;
 - ***b*** significa binário.

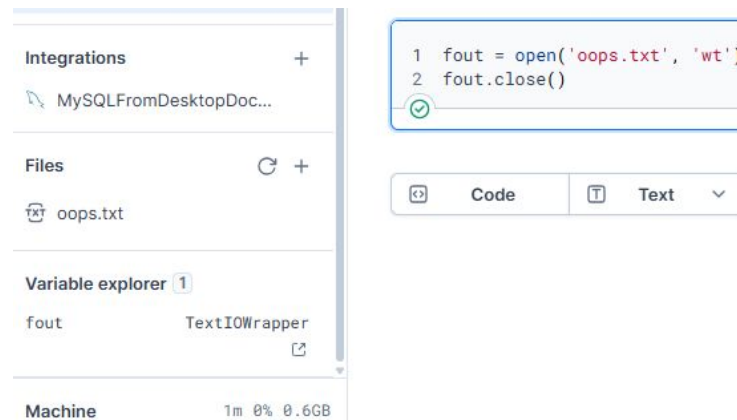
Manipulação de Arquivos em Disco

Criar ou Abrir com *open()*

Após abrir o arquivo, precisamos chamar funções para ler ou escrever dados e, posteriormente, precisamos fechar o arquivo para garantir que todas as gravações sejam concluídas e que a memória seja liberada. É comum usar o *with* para automatizar esse processo.

Este programa abre um arquivo chamado *oops.txt* e o fecha sem escrever nada, criando um arquivo vazio:

```
fout = open('oops.txt', 'wt')  
fout.close()
```



Manipulação de Arquivos em Disco

Escrevendo um Arquivo Texto com *print()*

Este comando sobrescreve o arquivo *oops.txt*. Usamos o argumento *file* para *print*. Sem ele, o comando *print* escreve na saída padrão.

```
fout = open('oops.txt', 'wt')
print('Oops, I created a file.', file=fout)
fout.close()
```

```
poem = '''There was a young lady named Bright,
        Whose speed was far faster than light;
        She started one day
        In a relative way,
        And returned on the previous night.'''
len(poem)
```

Escrevendo Arquivo Texto com *write()*

Também podemos usar o comando *write*. O código a seguir escreve o poema inteiro no arquivo *relativity* em uma única chamada:

```
fout = open('relativity', 'wt')
fout.write(poem)
fout.close()
```


Manipulação de Arquivos em Disco

Uso do *print()* VS *write()*

A principal diferença entre *print* e *write* em Python está no formato da saída, na formatação automática e no objetivo de cada uma. O *print* serve para exibir dados formatados na tela, enquanto o *write* grava dados brutos em arquivos. Principais diferenças:

- Finalidade: O *print()* é voltado para humanos (exibe informações legíveis no console). O *write()* é voltado para fluxos ou manipulação de arquivos (salva dados puros em disco, envia via rede, etc.);
- Quebra de linha: O *print()* adiciona uma quebra de linha (`\n`) automaticamente ao final do texto. O *write()* não adiciona nada, ele escreve exatamente o que é passado;
- Conversão de tipos: O *print()* converte automaticamente qualquer coisa passada para texto (números, listas, etc.). O *write()* aceita apenas strings, gerando um erro caso você tente escrever um número diretamente;
- Formatação: O *print()* permite formatações (como f-strings) e múltiplos argumentos separados por espaços. O *write()* escreve uma string estática ou concatenada.

Manipulação de Arquivos em Disco

Leitura de Arquivos Texto com *read()*, *readline()*, ou *readlines()*

Podemos chamar a função *read()* sem argumentos para ler o arquivo inteiro de uma só vez, como mostrado no exemplo a seguir (cuidado, pois o consumo de memória está diretamente ligado ao tamanho do arquivo):

```
poem = ''
fin = open('relativity', 'rt' )
chunk = 100
while True:
    fragment = fin.read(chunk)
    if not fragment:
        break
    poem += fragment
fin.close()
len(poem)
```

Podemos definir um número máximo de caracteres para limitar a quantidade de dados retornados pela função *read()*. No exemplo, lemos 100 caracteres por vez e adicionamos cada trecho a uma string de poema para reconstruir o original.

Ao final do arquivo, as chamadas subsequentes a *read()* retornam string vazia ("), que é tratada como *False* no fragmento *if not*. Isso interrompe o loop *while True*.

Manipulação de Arquivos em Disco

Leitura de Arquivos Texto com *read()*, *readline()*, ou *readlines()*

Podemos também ler o arquivo linha por linha usando a função *readline()*. No exemplo a seguir, adicionamos cada linha à string do poema para reconstruir o original.

```
poem = ''
fin = open('relativity', 'rt' )
while True:
    line = fin.readline()
    if not line:
        break
    poem += line
fin.close()
len(poem)
```

A função *readlines()* lê uma linha por vez e retorna uma lista de strings de uma única linha:

```
fin = open('relativity', 'rt' )
lines = fin.readlines()
fin.close()
print(len(lines), 'lines read')
for line in lines:
    print(line, end='')
```

Manipulação de Arquivos em Disco

Escrita e Leitura de Arquivo Binário com *write()* e *read()*

Se incluirmos um 'b' na string *mode*, o arquivo será aberto em modo binário. Nesse caso, você lê e grava bytes em vez de uma string:

```
bdata = bytes(range(0, 256))
fout = open('bfile', 'wb')
size = len(bdata)
offset = 0
chunk = 100
while True:
    if offset > size:
        break
    fout.write(bdata[offset:offset+chunk])
    offset += chunk
fout.close()
```

A leitura de arquivos binários é simples, bastando abrir com 'rb':

```
fin = open('bfile', 'rb')
bdata = fin.read()
fin.close()
len(bdata)
```

Manipulação de Arquivos em Disco

Fechando Arquivos de Automaticamente com Uso do *with*

- Se esquecermos de fechar um arquivo aberto, ele será fechado pelo python assim que não for mais referenciado, ou seja, se abrirmos um arquivo dentro de uma função e não o fecharmos explicitamente, ele será fechado quando a função terminar.
- Mas podemos ter aberto o arquivo em uma função de longa duração ou na seção principal do programa. Nesse caso, o arquivo deve ser fechado para garantir que quaisquer gravações restantes sejam concluídas.
- Python possui gerenciadores de contexto para limpar processos como arquivos abertos. Usamos a forma “*with <expression> as <variable>*”:

```
with open('relativity', 'wt') as fout:  
    fout.write(poem)
```

Manipulação de Arquivos em Disco

Mudança de Posição de Cursor com *seek()*

À medida que lemos e escrevemos, o python mantém o controle de onde estamos.

- A função *tell()* retorna o deslocamento atual a partir do início do arquivo, em bytes.
- A função *seek()* permite pulamos para outro deslocamento de byte no arquivo.

Isso significa que não precisamos ler todos os bytes para ler o último. Podemos usar *seek()* para ir até o último byte e ler apenas um byte.

```
# Open the binary file
fin = open('bfile', 'rb')
fin.tell() # Return 0, start of the file
# Jump to one byte before the end of the file
fin.seek(255)
# Read until the end of the file:
bdata = fin.read()
len(bdata)
```

A função *seek()* aceita um segundo argumento: *seek(offset, origin)*:

- Se *origin* for 0 (o padrão), o ponteiro será deslocado em offset bytes a partir da origem.
- Se *origin* for 1, o ponteiro será deslocado em offset bytes a partir da posição atual.
- Se *origin* for 2, o ponteiro será deslocado em offset bytes a partir do fim do arquivo.

Manipulação de Arquivos em Disco

Operações com Arquivos

Python modela suas operações com arquivos seguindo o padrão do Unix. Algumas funções, como *chown()* e *chmod()*, têm os mesmos nomes, mas existem algumas outras novas:

```
# Check Existence with exists()
import os
os.path.exists('oops.txt')
# Check Type with isfile()
import os
name = 'oops.txt'
os.path.isfile(name)
os.path.isdir('.')
# Copy with copy() -- copy files
import shutil
shutil.copy('oops.txt', 'ohno.txt')
```

```
# Move with move() -- make a copy with the new
name and then remove the prior
import shutil
shutil.move('oops.txt', 'ohno.txt')
# Change name with rename()
import os
os.rename('ohno.txt', 'ohwell.txt')
```

Manipulação de Arquivos em Disco

Operações com Arquivos

Python modela suas operações com arquivos seguindo o padrão do Unix. Algumas funções, como *chown()* e *chmod()*, têm os mesmos nomes, mas existem algumas outras novas:

```
# Create links with link() or symlink()
import os
os.link('oops.txt', 'yikes.txt')
os.path.isfile('yikes.txt')
os.symlink('oops.txt', 'jeepers.txt')
os.path.islink('jeepers.txt')
# Change Permissions with chmod()
import os
os.chmod('oops.txt', 0o400)
```

```
# Change Ownership with chown()
uid = 5
gid = 22
os.chown('oops', uid, gid)
# Delete a File with remove()
os.remove('oops.txt')
os.path.exists('oops.txt')
```


Manipulação de arquivos e JSON

PARTE 2

Manipulação de Dados no Formato JSON

Manipulação de Dados no Formato JSON

O Que É o JSON

- Antes do JSON se tornar o padrão da indústria, o XML era amplamente utilizado, muitas vezes em conjunto com tecnologias como o AJAX (Asynchronous JavaScript and XML). O XML é o principal predecessor e antecessor direto do JSON na troca de dados entre aplicações web é o XML;
- JSON (*JavaScript Object Notation*) é o formato de troca de dados mais utilizado atualmente — APIs REST, arquivos de configuração, bancos NoSQL e logs estruturados usam JSON amplamente. Python tem suporte nativo via módulo json, sem necessidade de instalar nada;
- O JSON é superior para a maioria das aplicações modernas porque é mais leve, mais rápido de processar e nativamente compatível com linguagens de programação web. Ele evita a redundância de marcações, tornando a transferência de dados em APIs muito mais eficiente.

Manipulação de Dados no Formato JSON

JSON VS XML

```
{  
  "palestrante": {  
    "nome": "Ana Lima",  
    "vip": true,  
    "linguagens": [  
      "Python",  
      "SQL"  
    ]  
  }  
}
```

```
<palestrante vip="true">  
  <nome>Ana Lima</nome>  
  <linguagens>  
    <item>Python</item>  
    <item>SQL</item>  
  </linguagens>  
</palestrante>
```

Manipulação de Dados no Formato JSON

Módulo json

Para importar e fazer uso do módulo: *import json*

Existem quatro funções centrais, organizadas em dois pares:

- *dumps* e *loads* trabalham com strings
- *dump* e *load* trabalham com arquivos

json.dumps() converte um objeto Python em uma string JSON.

json.loads() converte uma string JSON de volta em objeto Python.

json.dump() escreve em um arquivo.

json.load() faz a leitura de um arquivo.

Manipulação de Dados no Formato JSON

Mapeamento de Tipos - Python e JSON

Python	JSON
<code>dict</code>	<code>object {}</code>
<code>list</code> , <code>tuple</code>	<code>array []</code>
<code>str</code>	<code>string</code>
<code>int</code> , <code>float</code>	<code>number</code>
<code>True</code> / <code>False</code>	<code>true</code> / <code>false</code>
<code>None</code>	<code>null</code>

Manipulação de Dados no Formato JSON

Serializando com Python e JSON

```
import json

dados = {
    "nome": "Marcelo",
    "idade": 40,
    "linguagens": ["Python", "C#", "SQL"],
    "ativo": True,
    "salario": None # None vira null no JSON
}

# Conversão básica
texto = json.dumps(dados)
print(type(texto)) # <class 'str'>
print(texto)
```

```
# Com formatação legível (indentação)
texto_formatado = json.dumps(dados, indent=4,
                             ensure_ascii=False)
print(texto_formatado)
```

```
# Ordenar chaves alfabeticamente
texto_ordenado = json.dumps(dados, indent=4,
                             sort_keys=True)
```

```
# ensure_ascii=False → preserva caracteres
especiais (acentos)
texto_acentos = json.dumps({"cidade": "São
                             Paulo"}, ensure_ascii=False)
print(texto_acentos) # {"cidade": "São Paulo"}
```

Manipulação de Dados no Formato JSON

Desserialização com Python e JSON

```
import json

texto = '{"nome": "Marcelo", "idade": 40, "ativo": true, "salario": null}'
dados = json.loads(texto)

print(type(dados))          # <class 'dict'>
print(dados["nome"])        # Marcelo
print(dados["ativo"])       # True  (bool Python)
print(dados["salario"])     # None  (não null)

# JSON array → lista Python
lista_json = '[1, 2, 3, "quatro", true]'
lista = json.loads(lista_json)
print(type(lista))         # <class 'list'>
print(lista)               # [1, 2, 3, 'quatro', True]
```

Manipulação de Dados no Formato JSON

Escrita em Arquivo

```
import json

dados = {
    "conferencia": "SurgeTech",
    "ano": 2025,
    "palestrantes": [
        {"nome": "Ana Lima", "empresa": "Tech Solutions"},
        {"nome": "Bruno Costa", "empresa": "Data Labs"}
    ],
    "online": False
}

# Escrever em arquivo
with open("conferencia.json", "w", encoding="utf-8") as f:
    json.dump(dados, f, indent=4, ensure_ascii=False)
```


Manipulação de Dados no Formato JSON

Leitura de Arquivo

```
import json

# Ler arquivo JSON
with open("conferencia.json", "r", encoding="utf-8") as f:
    dados = json.load(f)

print(type(dados))                # <class 'dict'>
print(dados["conferencia"])        # SurgeTech
print(dados["palestrantes"][0]["nome"]) # Ana Lima

# Iterar sobre lista dentro do JSON
for palestrante in dados["palestrantes"]:
    print(f'{palestrante["nome"]} - {palestrante["empresa"]}')

```

Manipulação de Dados no Formato JSON

Tratamento de Erros

```
# — JSON inválido —————
texto_invalido = '{"nome": "Marcelo", "idade": }' # JSON malformatado
try:
    dados = json.loads(texto_invalido)
except json.JSONDecodeError as e:
    print(f"Erro ao decodificar JSON: {e}")
    print(f"Linha: {e.lineno}, Coluna: {e.colno}")

# — Arquivo não encontrado —————
try:
    with open("inexistente.json", "r") as f:
        dados = json.load(f)
except FileNotFoundError:
    print("Arquivo JSON não encontrado.")
```

Manipulação de Dados no Formato JSON

Tratamento de Erros

```
# — Chave inexistente - usar .get() como proteção —————
dados = {"nome": "Marcelo"}
idade = dados.get("idade", 0)      # retorna 0 se não existir
email = dados.get("email", None)   # retorna None se não existir

# — Tipo não serializável —————
from datetime import datetime
evento = {"data": datetime.now()}
try:
    json.dumps(evento)
except TypeError as e:
    print(f"Tipo não serializável: {e}")
    # Object of type datetime is not JSON serializable
```

Manipulação de Dados no Formato JSON

Serialização com Tipos Customizados

Por padrão, `json.dumps()` não sabe converter objetos como `datetime`, `Decimal` ou classes customizadas. Existem duas soluções.

```
import json
from datetime import datetime
from decimal import Decimal

# Uso de funcao default
def serializar_customizado(obj):
    if isinstance(obj, datetime):
        return obj.isoformat()
    if isinstance(obj, Decimal):
        return float(obj)
    raise TypeError(f"Tipo não serializável")
```

```
evento = {
    "nome": "SurgeTech",
    "data_inicio": datetime(2025, 6, 10, 8, 0),
    "preco": Decimal("299.90")
}

texto = json.dumps(evento,
    default=serializar_customizado, indent=2)
print(texto)
```

Manipulação de Dados no Formato JSON

Serialização com Tipos Customizados

Por padrão, `json.dumps()` não sabe converter objetos como `datetime`, `Decimal` ou classes customizadas. Existem duas soluções: uso de função default ou customizar o `JSONEncoder`.

```
import json
from datetime import datetime
from decimal import Decimal

# Uso de funcao default
def serializar_customizado(obj):
    if isinstance(obj, datetime):
        return obj.isoformat()
    if isinstance(obj, Decimal):
        return float(obj)
    raise TypeError(f"Tipo não serializável")
```

```
evento = {
    "nome": "SurgeTech",
    "data_inicio": datetime(2025, 6, 10, 8, 0),
    "preco": Decimal("299.90")
}

texto = json.dumps(evento,
    default=serializar_customizado, indent=2)
print(texto)
```

Manipulação de Dados no Formato JSON

Serialização com Tipos Customizados

Por padrão, `json.dumps()` não sabe converter objetos como *datetime*, *Decimal* ou classes customizadas. Existem duas soluções: uso de função default ou customizar o `JSONEncoder`.

```
import json
from datetime import datetime

# Customizando o JSONEncoder
class EncoderCustomizado(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime):
            return {
                "__type__": "datetime",
                "valor": obj.isoformat()
            }
        return super().default(obj)
```

```
dados = {"criado_em": datetime.now()}
texto = json.dumps(
    dados,
    cls=EncoderCustomizado,
    indent=2
)
print(texto)
```

Manipulação de Dados no Formato JSON

Atualizando Arquivo JSON Existente

```
import json

# Ler, modificar e salvar
with open("conferencia.json", "r", encoding="utf-8") as f:
    dados = json.load(f)

# Modificar
dados["ano"] = 2026
dados["palestrantes"].append({
    "nome": "Carla Mendes",
    "empresa": "Cloud Nexus"
})

# Salvar de volta
with open("conferencia.json", "w", encoding="utf-8") as f:
    json.dump(dados, f, indent=4, ensure_ascii=False)
```

Manipulação de Dados no Formato JSON

Múltiplos Objetos por Arquivo - JSON Lines

Formato comum em logs e pipelines de dados, onde cada linha é um JSON independente.

```
import json

registros = [
    {"id": 1, "evento": "login", "usuario": "ana"},
    {"id": 2, "evento": "logout", "usuario": "bruno"},
    {"id": 3, "evento": "login", "usuario": "carla"},
]

# Escrever JSON Lines
with open("eventos.jsonl", "w", encoding="utf-8") as f:
    for registro in registros:
        f.write(json.dumps(registro, ensure_ascii=False) + "\n")

# Ler JSON Lines
with open("eventos.jsonl", "r", encoding="utf-8") as f:
    for linha in f:
        registro = json.loads(linha.strip())
        print(registro["usuario"], "-", registro["evento"])
```


Manipulação de Dados no Formato JSON

Comparação com Outras Bibliotecas

Para casos mais avançados, bibliotecas externas complementam o módulo nativo.

```
# ujson - mais rápido que o módulo
# nativo para grandes volumes
import ujson
dados = ujson.loads(texto)
texto = ujson.dumps(dados)

# orjson - o mais rápido disponível,
# suporta datetime nativamente
import orjson
texto = orjson.dumps(dados)
dados = orjson.loads(texto)
```

```
# pydantic - validação de schema com tipos
from pydantic import BaseModel

class Palestrante(BaseModel):
    nome: str
    empresa: str
    vip: bool = False

p = Palestrante(**{"nome": "Ana", "empresa":
    "Tech"})
print(p.model_dump_json())
```

Manipulação de Dados no Formato JSON

Considerações ao Lidar com Arquivos JSON

Em Python, para salvar ou processar JSON contendo caracteres não ASCII (como acentos ou símbolos especiais) como texto legível em vez de sequências escapadas (por exemplo, `\u00e9`), você deve usar o parâmetro `ensure_ascii=False`.

```
import json

data = {"mensagem": "Olá, como vai?"}

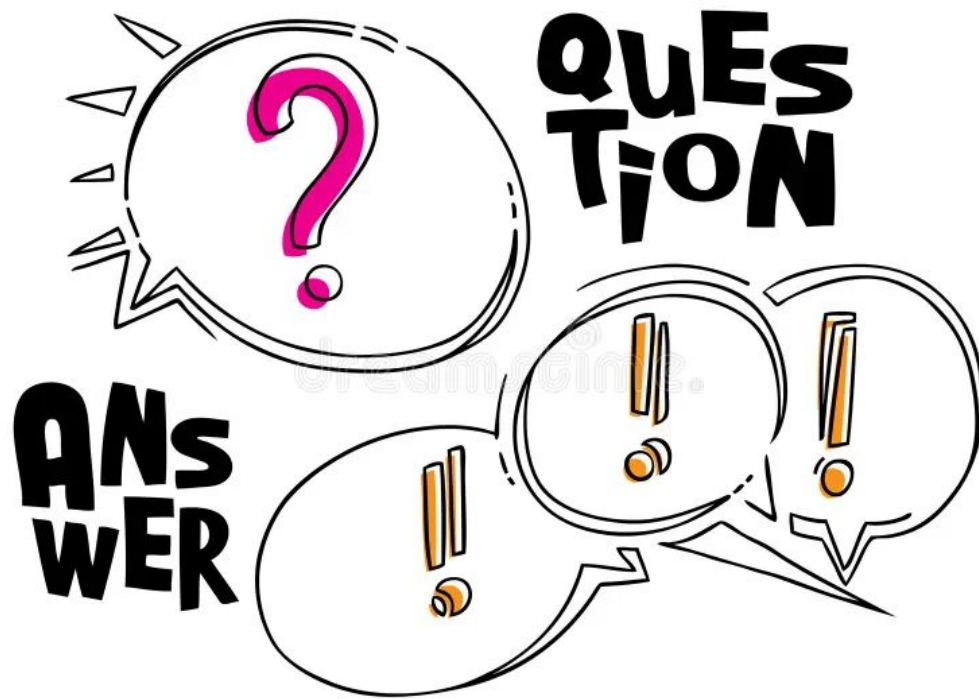
# The 'encoding' here ensures the file itself is saved as UTF-8
with open('data.json', 'w', encoding='utf-8') as f:
    # ensure_ascii=False keeps the actual characters instead of codes like \u00e9
    json.dump(data, f, ensure_ascii=False, indent=4)
```

Manipulação de Dados no Formato JSON

Resumo das Funções de Python para JSON

Função	Entrada	Saída	Uso
<code>json.dumps()</code>	objeto Python	string JSON	serializar para string
<code>json.loads()</code>	string JSON	objeto Python	deserializar de string
<code>json.dump()</code>	objeto Python + arquivo	arquivo JSON	escrever em arquivo
<code>json.load()</code>	arquivo JSON	objeto Python	ler de arquivo

Dúvidas e Perguntas



Referências Bibliográficas

- Get Programming, Ana Bell, Manning Publications;
- Introducing Python, Bill Lubanovic, O'Reilly Media, Inc;
- Learn to Code by Solving Problems, Daniel Zingaro, No Starch Press queue;
- Python Crash Course, Eric Matthes, No Starch Press;
- Streamlining Your Research Laboratory with Python, Mark F. Russo, William Neil, Wiley;
- The Quick Python Book, Naomi Ceder, Manning Publications;
- <https://deepnote.com/docs/incoming-connections>;
- <https://ai.google.dev/gemini-api/docs>;
- <https://developers.openai.com/api/docs/libraries>.